

What is Next.Js?

Next.js is a React-based framework used to build fast, SEO-friendly, and full-stack web applications.

It provides features like:

- Server-Side Rendering (SSR)
- Static Site Generation (SSG)
- API Routes
- File-based Routing
- Image Optimization
- Middleware
- Full-stack support

It is built on top of React by Vercel.

Difference between SPA and Next.js framework?

SPA (Single Page Application) loads a single HTML page and dynamically updates content using JavaScript without refreshing the page. React applications usually work as SPAs.

Next.js is a React framework that supports SPA features along with Server-Side Rendering, Static Site Generation, routing, API handling, and SEO optimization. Unlike a pure SPA, Next.js provides better performance and SEO because pages can be rendered on the server.

What is JSX?

JSX is a syntax used in React that allows us to write HTML-like code inside JavaScript.

What is app route in next js?

App Router in Next.js is the modern routing system introduced in Next.js 13 that uses the app directory for routing and supports layouts, server components, loading states, and nested routing for better performance and scalability.

```
my-app/  
├── app/  
│   ├── page.js  
│   ├── about/  
│   │   └── page.js  
│   └── contact/  
│       └── page.js
```

What is layout.jsx file in Next.Js?

layout.jsx in Next.js App Router is a special file used to create shared layouts for multiple pages. It wraps pages using the children prop and is commonly used for navbars, sidebars, and footers.

```
app/
├── layout.jsx
├── page.jsx
├── about/
│   └── page.jsx
```

What is the nested layout in Next.JS?

Nested Layout in Next.js is a feature of the App Router where layouts can be created inside folders to provide different shared UI structures for different sections of the application.

```
app/
├── layout.jsx
├── page.jsx
├── dashboard/
│   ├── layout.jsx
│   ├── page.jsx
│   └── analytics/
│       └── page.jsx
```

Route	Layout
/	Navbar + Footer
/dashboard	Navbar + Sidebar + Dashboard
/admin	Admin Sidebar

What is Browser rendering feature and why we use "use client" in Next.Js?

Browser-side rendering features are functionalities that require the browser environment, such as event handlers, React hooks, and browser APIs like window or localStorage. These features require "use client" in Next.js App Router.

Feature	Need "use client"?
useState	✓
useEffect	✓
onClick	✓
onChange	✓
window	✓
document	✓
localStorage	✓
Static JSX	✗
async fetch	✗
SEO content	✗

When should we use "use client"?

When we are using Hooks event and browser API then we should to make component client component using "use client"

What is cache: 'no-store' in Next.js?

cache: 'no-store' is a fetch option in Next.js that disables caching. It forces Next.js to fetch fresh data on every request, making the route dynamically rendered. It is commonly used for dashboards, user-specific data, and real-time information where up-to-date data is required.

```
const res = await fetch('https://api.example.com/users', {
  cache: 'no-store',
});
```

SSR (Server Side Rendering):

HTML is generated on every request at the server. It provides fresh data but is slower than static pages.

```
User Request
  ↓
Server fetches latest data
  ↓
Server generates HTML
```

↓
HTML sent to browser

SSG (Static Site Generation):

HTML is generated during build time and served as static files. It is very fast but data updates require a new build.

npm run build
↓
HTML generated
↓
Stored on server/CDN
↓
Users get same static HTML

ISR (Incremental Static Regeneration):

ISR is a combination of SSG and SSR. The page is generated at build time but can be automatically updated after a specific time without rebuilding the entire application.

Build Time
↓
Static page generated
↓
Users get cached page

After 60 sec
↓
Next request triggers regeneration
↓
New page cached

CSR (Client-Side Rendering)

In CSR, the browser is responsible for creating and displaying the page. The server sends JavaScript files, and the browser fetches data and renders the UI.

User Request
↓
Server fetches data

↓
Server generates HTML
↓
HTML sent to Browser
↓
Page displayed

Why fetch data on the server side?

- Faster initial page load
- Better SEO
- API keys remain secure
- Less JavaScript sent to browser
- Direct database access possible

```
export default async function UsersPage() {  
  const res = await fetch("https://api.example.com/users");  
  const users = await res.json();  
  
  return (  
    <div>  
      {users.map(user => (  
        <p key={user.id}>{user.name}</p>  
      ))}  
    </div>  
  );  
}
```